Power7 - E-Commerce Project

Team: Ponoran Claudiu-Mihai, Oprea Iulian-Marian

Presentation 2: Specification & Prototype

1. Mandatory Requirements (A1-A12)

- **A1 Product catalog:** will use Entity Framework Core to fetch products and their associated categories from the SQLite database, returning them via a controller to the frontend (max 30 items per page)

- **A2 Basic search:** a search bar in the UI will send a GET request with a query string parameter to the API then the ASP.NET backend will use a Contains LINQ query on the Name attribute to return matching products or a specific "no results found"

- **A3 Shopping cart:** the cart state will be stored persistently in the Cart_Item database table associated with the user_id for authenticated users. AJAX requests will handle adding, updating, and removing items, with the server strictly validating requested quantities against the available database stock

- **A4 User accounts:** will implement registration and login using ASP.NET Core Identity, storing credentials in the User table

- **A5 Checkout:** a multi-step frontend form will collect the user's selected address and delivery method, calculating the final total on the server before initializing a Stripe Checkout session and displaying the pending order number

- **A6 Sandbox payment:** Stripe Checkout in test mode will handle the card input securely, an ASP.NET API endpoint will listen for the Stripe Webhook (checkout.session.completed) to asynchronously and securely mark the payment_status as "Paid"

- **A7 Stock management:** upon receiving the successful payment webhook from Stripe, a database transaction using pessimistic concurrency control will decrement the purchased quantities from the Product table and restore them if an order is cancelled

- **A8 Order history:** a protected backend route will query the Order and Order_Item tables filtered by the current user's ID, displaying the date, purchased products, total cost, and current status on a dedicated frontend view

- **A9 Administration panel:** will build protected endpoints and views restricted to users with the "Admin" role, allowing them to perform CRUD operations on products/categories (including image uploads) and update the order_status flow

- **A10 Basic security:** will utilize ASP.NET Core built-in features to enforce Server-Side Model Validation, Anti-Forgery Tokens for CSRF protection on forms, and the [Authorize(Roles="Admin")] attribute to block regular users from accessing admin routes on the server

- **A11 Responsive design:** the frontend will be constructed with Bootstrap 5's responsive grid system and utility classes, ensuring all UI elements scale properly and remain usable

- **A12 Automated tests:** will write at least 10 unit and integration tests using xUnit, specifically targeting the cart total calculation logic, stock deduction, and mocking the payment confirmation process, executable via a single dotnet test command

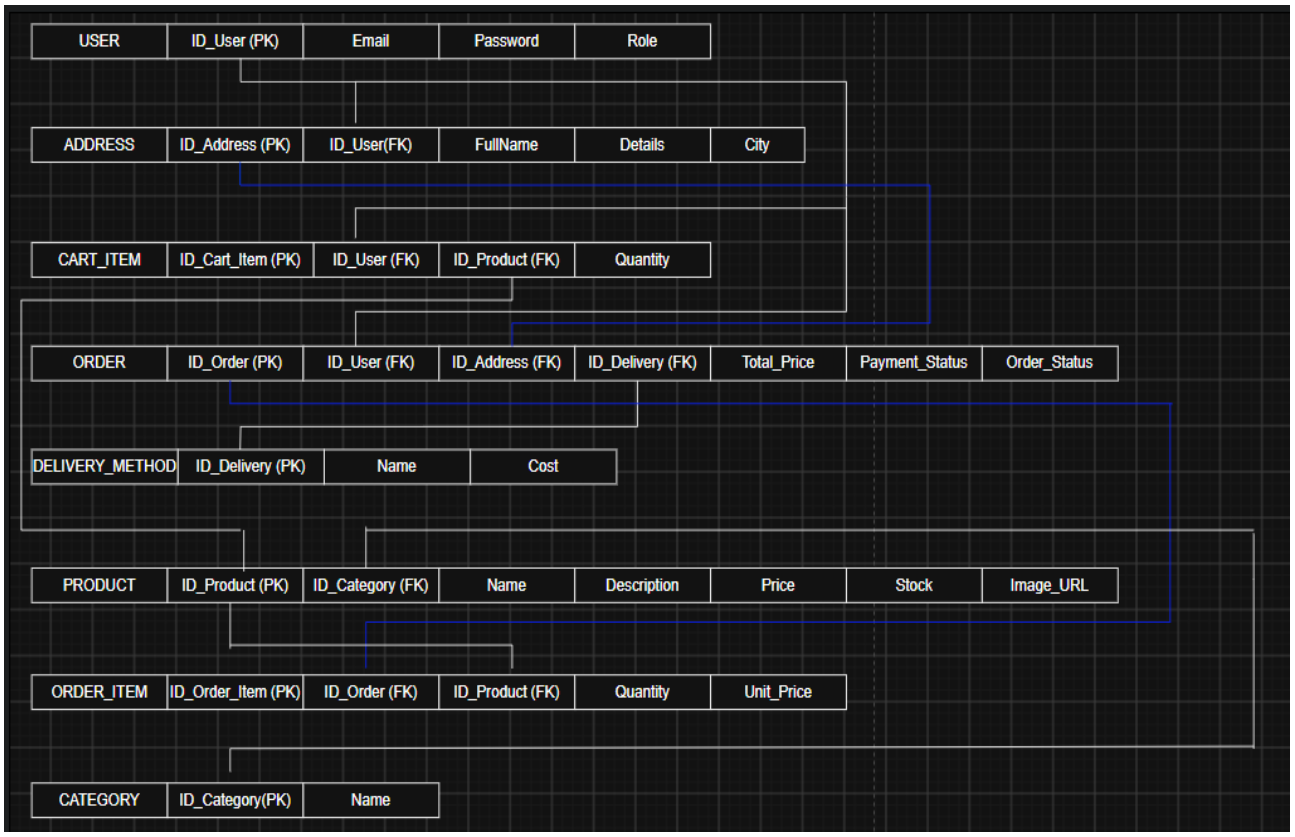
2. Selected Bonus Features

- **B1 Filtered search:** will implement combinable filters (by category and price range) and sorting options, passing these parameters via URL query strings to the backend

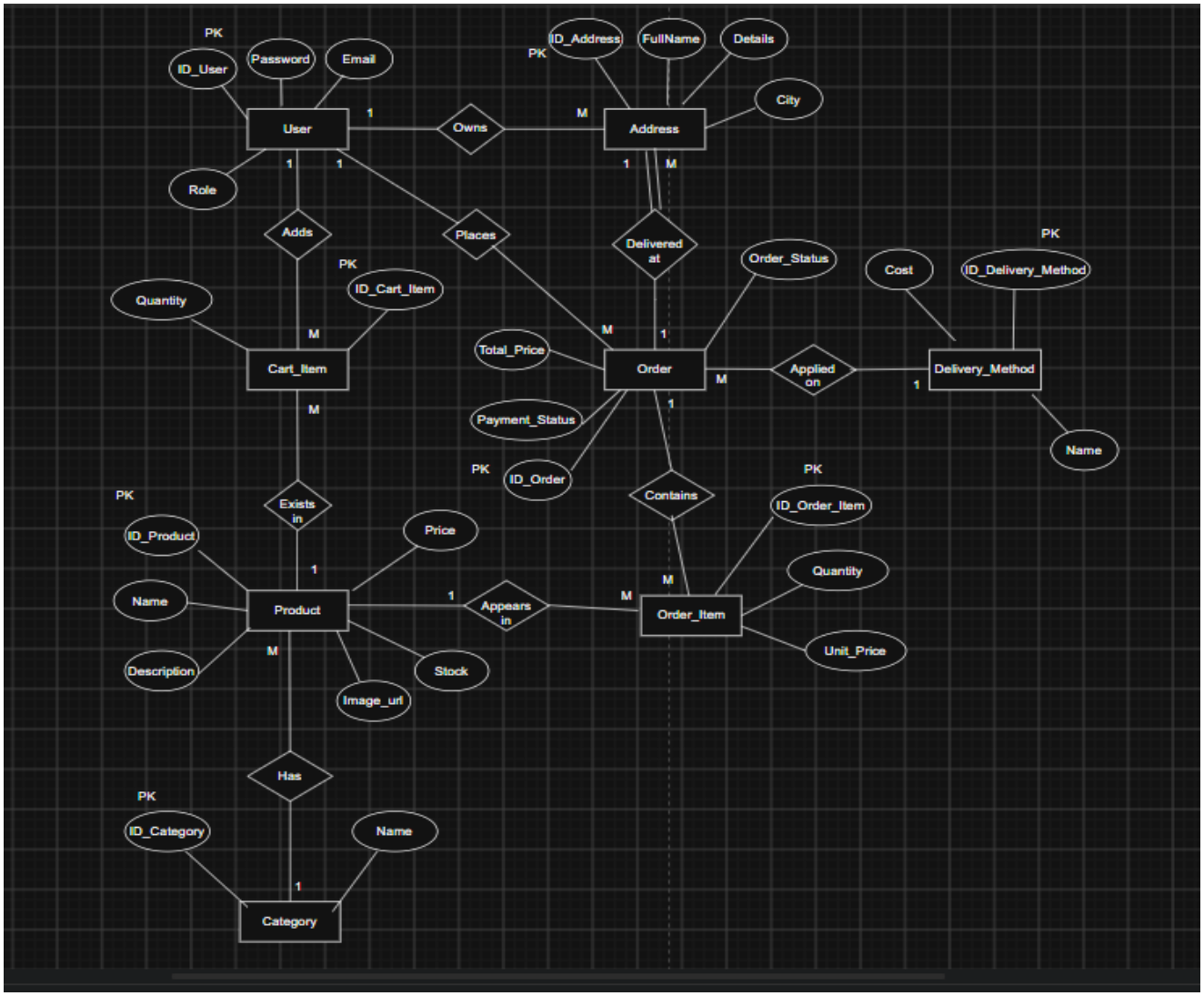
- **B6 Transactional email:** will integrate Mailtrap within the backend webhook listener to automatically trigger an order confirmation email to the user once the payment is successfully verified

3. Diagrams and Architecture

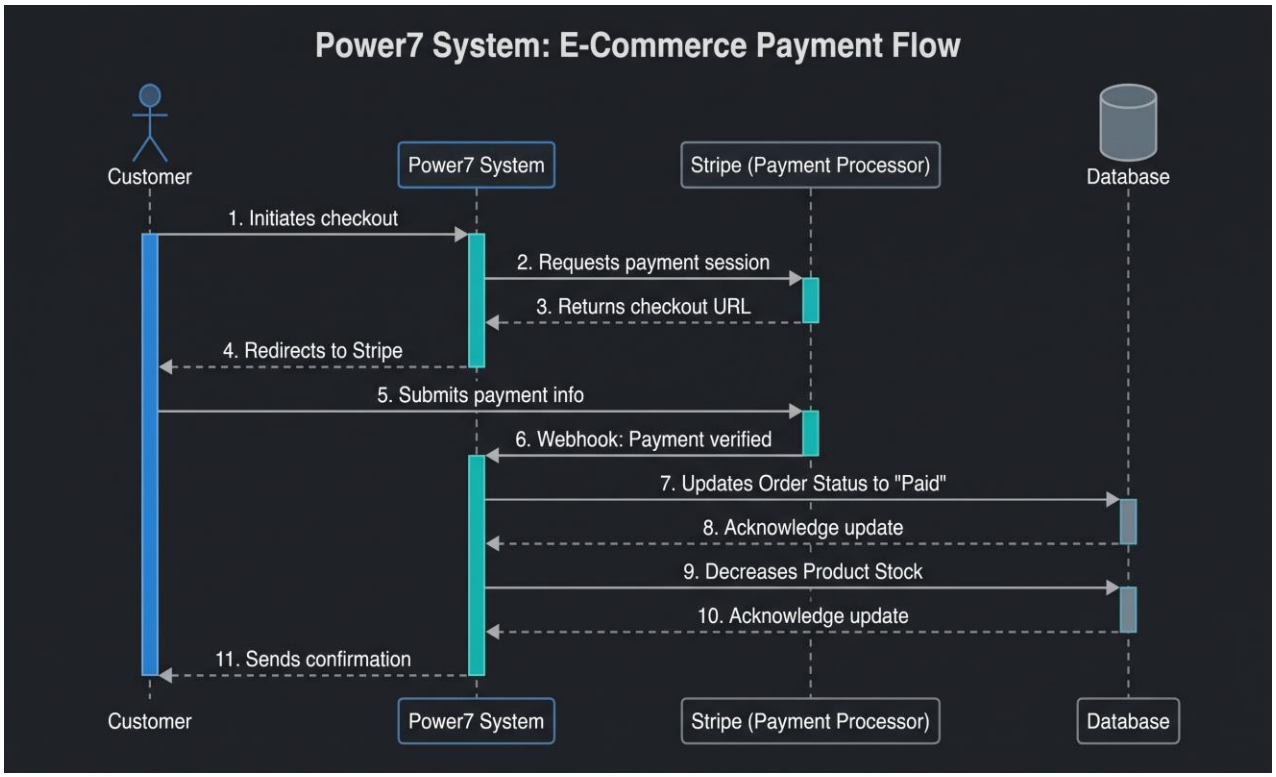
A. **Relational Model - b**lue lines are used to avoid confusion when some of the drawn lines meet



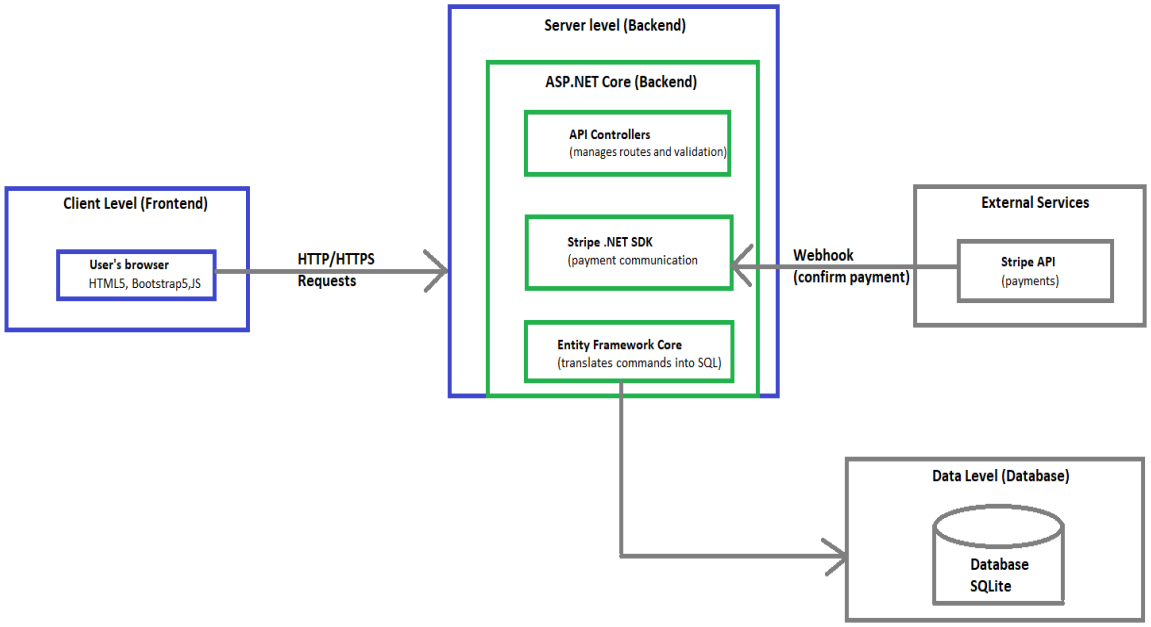
B. Entity-Relationship Diagram



C. Critical Flow



D. Component Diagram

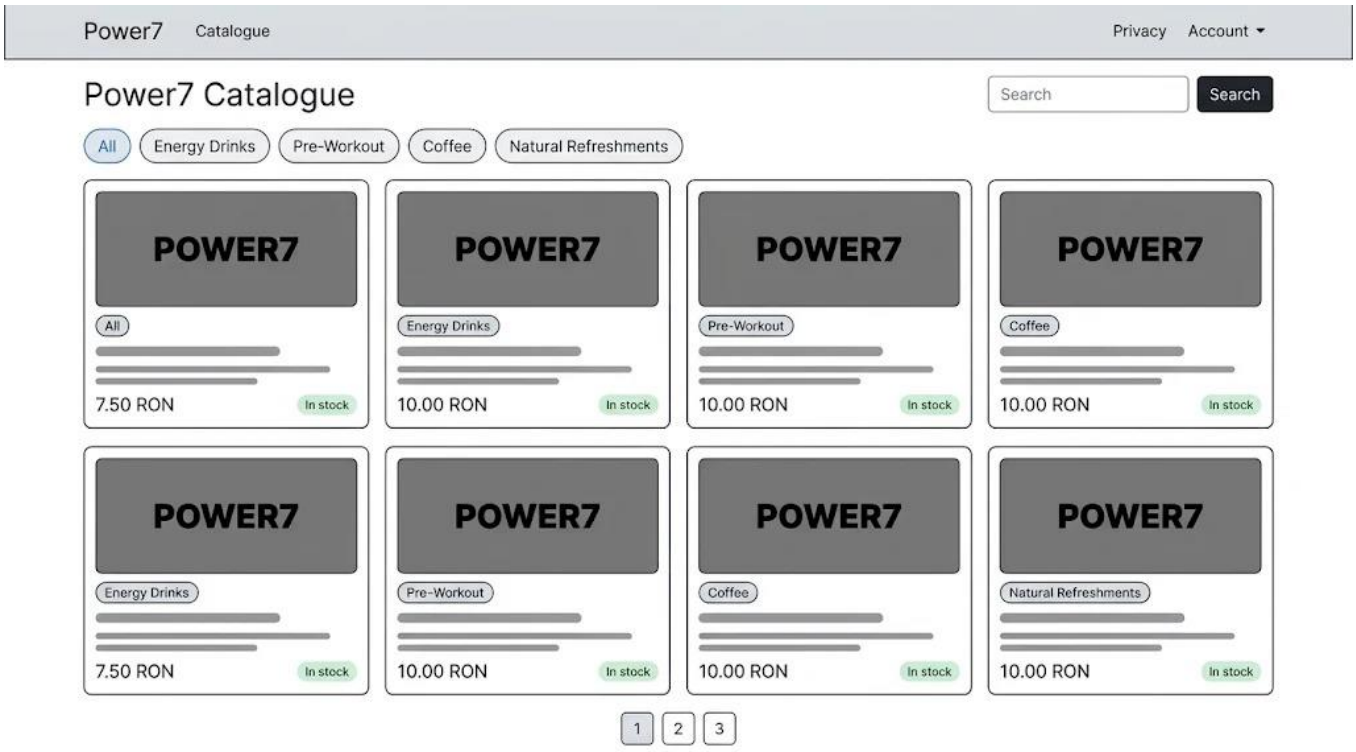


4. API Endpoints

HTTP Method	API Route	Acces Level	Short Description
GET	/api/products	Public	Returns the catalogue and the filters (A1, A2)
GET	/api/products/{id}	Public	The details of a single product (A1)
POST	/api/auth/register	Public	Register new account (A4)
POST	/api/auth/login	Public	Authentication and generation of token/session (A4)
GET	/api/cart	Client	Visualize cart (A3)
POST	/api/cart/items	Client	Add a product in cart (A3)
POST	/api/checkout/session	Client	Initiates the Stripe payment session (A5)
POST	/api/webhook/stripe	Public (Stripe)	Stripe sends the confirmation of the payment (A6, A7)
GET	/api/orders/history	Client	Current user's order history (A8)
POST	/api/admin/products	Admin	Add a new product in the catalogue (A9)
PUT	/api/admin/products/{id}	Admin	Product/stock editing (A9)
PUT	/api/admin/orders/{id}/status	Admin	Order status changement (A9)

5. Wireframes

A. Wireframe for Catalogue



B. Wireframe for Login/Register

